Assignment 3

Fernando Melo Filho
fmelofilho2950@floridapoly.edu

CAP 5410 — Computer Vision
Assignment 3

Instructor: Dr. Muhammad Abid

Convolutional Neural Networks for MNIST Digit Classification

Department of Computer Science
Florida Polytechnic University, Florida, USA

October 2026

CONTENTS

CONTENTS ii

LIST OF FIGURES iv

LIST OF TABLES v

1 INTRODUCTION 1

1.1 OBJECTIVES 1

1.2 BACKGROUND 2

1.2.1 Training. 2

1.2.2 Evaluation Metrics 2

1.3 REPORT ORGANIZATION 2

2 DATASET AND EXPERIMENTAL SETUP 4

2.1 DATASET AND DATA SPLIT 4

2.2 RANDOM SEED AND IDENTICAL STARTING CONDITIONS 5

3 TASK 1 — CNN IMPLEMENTATION AND MODEL DOCUMENTATION 6

3.1 ARCHITECTURE AND LAYER DIMENSIONS 6

3.2 TRAINABLE PARAMETERS 7

3.3 TRAINING CONFIGURATION 8

3.4 TRAINING LOOP AND METRIC CALCULATION 9

4 TASK 2 — FIRST-EPOCH BATCH-LEVEL LEARNING ANALYSIS 12

5 TASK 3 — TEN-EPOCH TRAINING AND VALIDATION ANALYSIS 15

6 TASK 4 — CONTROLLED LEARNING-RATE EXPERIMENT 18

6.1 EXPERIMENT DESIGN 18

6.2 RESULTS AND SELECTION 19

7 TASK 5 — INDEPENDENT TESTING AND REPRODUCIBILITY 22
7.1 TEST-SET EVALUATION 22
7.2 EXECUTION ENVIRONMENT AND LIBRARY VERSIONS 23
7.3 INSTRUCTIONS TO REPRODUCE THE EXPERIMENTS 23

8 CONCLUSION 25

REFERENCES 26

APPENDIX A SOURCE CODE 27

LIST OF FIGURES

Figure 4.1 Epoch 1: training loss by batch. 13
Figure 4.2 Epoch 1: training accuracy by batch. 13
Figure 4.3 Epoch 1: macro-F1 by batch. 14

Figure 5.1 Training and validation loss over 10 epochs. 16
Figure 5.2 Training and validation accuracy over 10 epochs. 16
Figure 5.3 Training and validation macro-F1 over 10 epochs. 17

Figure 6.1 Learning-rate comparison: loss and accuracy curves. 19
Figure 6.2 Learning-rate comparison: first-epoch loss. 19
Figure 6.3 Learning-rate comparison: validation accuracy (zoom). 20

LIST OF TABLES

Table 3.1 Trainable parameters per layer. 8
Table 3.2 Baseline training configuration. 9
Table 5.1 Baseline epoch-level metrics. 15
Table 6.1 Learning-rate comparison summary. 20
Table 7.1 Test-set results. 23
Table 7.2 Software and hardware environment. 23

1 INTRODUCTION

In Assignment 2 I applied box, Sobel and Gaussian filters, whose kernels are fixed by formula. A convolutional neural network (CNN) uses the same sliding-window operation, but it learns the kernel values from labeled examples (Goodfellow et al., 2016). In this assignment I implemented a small CNN in PyTorch (Paszke et al., 2019), trained it to recognize handwritten digits from the MNIST dataset, followed how it learns batch by batch and epoch by epoch, and ran a controlled experiment on the learning rate.

1.1 OBJECTIVES

The assignment has five tasks:

1. Implement the CNN in PyTorch and document every layer, its input/output dimensions, its trainable parameters and the training configuration.
2. Train the baseline for 10 epochs and plot the training loss, accuracy and F1-score batch by batch during the first epoch.
3. Plot training and validation loss, accuracy and F1-score over the 10 epochs and discuss convergence and over/underfitting.
4. Change one hyperparameter (I chose the learning rate), train the alternatives under the same conditions, and pick the best configuration using validation data only.
5. Evaluate the chosen model once on the reserved test set and document how to reproduce every result.

1.2 BACKGROUND

1.2.1 Training

The network outputs 10 scores (logits), one per digit. The cross-entropy loss turns them into probabilities with softmax and penalizes a low probability on the correct class:

$$\mathcal{L} = -\frac{1}{N}\sum_{n=1}^{N} \log \frac{e^{z_{n,y_n}}}{\sum_{c=0}^{9} e^{z_{n,c}}} \tag{1.1}$$

where $z_{n,c}$ is the score of image n for class c and y_n is its true label. Training repeats four steps for every mini-batch: forward pass, loss, back-propagation of the gradients, and a parameter update (Abid, 2026). With stochastic gradient descent (SGD) with momentum μ, the update for parameters θ with learning rate η is

$$v \leftarrow \mu v + \nabla_\theta \mathcal{L}, \qquad \theta \leftarrow \theta - \eta\, v \tag{1.2}$$

The learning rate controls the step size, which is why it is the factor I study in Task 4.

1.2.2 Evaluation Metrics

Accuracy is the fraction of correctly classified images. For each class c, precision P_c is the fraction of images predicted as c that really are c, and recall R_c is the fraction of images of class c that the model finds. The F1-score is their harmonic mean, $F1_c = 2P_cR_c/(P_c + R_c)$. Throughout this report I use *macro* averages: the metric is computed for each digit separately and then averaged with equal weight over the set of digits C,

$$\text{macro-F1} = \frac{1}{|C|}\sum_{c\in C} F1_c \tag{1.3}$$

so a digit the model handles badly lowers the score even if it is rare. On the full training, validation and test sets, C is all 10 digits. All three are computed with scikit-learn (Pedregosa et al., 2011) (`average="macro"`).

1.3 REPORT ORGANIZATION

Chapter 2 describes the dataset, the data split and the seeding shared by all tasks. Each of the five tasks then has its own chapter:

- Chapter 3: Task 1, CNN implementation and model documentation;
- Chapter 4: Task 2, first-epoch batch-level learning analysis;
- Chapter 5: Task 3, ten-epoch training and validation analysis;

- Chapter 6: Task 4, controlled learning-rate experiment;
- Chapter 7: Task 5, independent testing and reproducibility.

Chapter 8 summarizes the results and their limitations, and Appendix A lists the complete source code.

2 DATASET AND EXPERIMENTAL SETUP

The setup below applies to all five tasks. The code is split into five Python files. `dataloader.py` downloads MNIST and builds the data split, `model.py` defines the CNN, `train.py` contains the training and evaluation loops, and `plots.py` draws every figure. `main.py` runs all five tasks in order and writes a log of the program output to `results/output_log.txt`. I started from the "CNN for MNIST" example in the lecture notes (Abid, 2026), which is based on the PyTorch tutorials (PyTorch Team, 2026). I adapted it to the 28×28 MNIST images and added the validation split, the metrics and the experiment.

2.1 DATASET AND DATA SPLIT

MNIST (K., 2017; LeCun et al., 1998) contains 28×28 grayscale images of the handwritten digits 0–9: 60,000 for training and 10,000 for testing. I load it with `torchvision.datasets.MNIST`, which provides the same files as the Kaggle version. The 60,000 training images are split once into 48,000 for training (80%) and 12,000 for validation (20%) using a random permutation drawn from a generator seeded with 42 (Listing 2.1). Every experiment calls the same function with the same seed, so all of them use the exact same split. The 10,000 test images are not used for anything until Task 5.

```
def split_indices(seed=SPLIT_SEED):
    """Return (train_idx, val_idx): a fixed random 80/20 split of the
       60,000 images."""
    g = torch.Generator().manual_seed(seed)
    perm = torch.randperm(N_TRAIN + N_VAL, generator=g)
    return perm[:N_TRAIN].tolist(), perm[N_TRAIN:].tolist()


def train_mean_std(train_idx):
    """Mean and standard deviation of the pixels in the training split
       only.

    The validation and test images are not used here, so no information
    about them leaks into the preprocessing.
    """
    raw = torchvision.datasets.MNIST(root=DATA_DIR, train=True, download=
       True)
```

```
    pixels = raw.data[train_idx].float() / 255.0
    return pixels.mean().item(), pixels.std().item()
```

Listing 2.1. Fixed 80/20 split and normalization statistics (`dataloader.py`).

Preprocessing. Each image goes through two transforms: `ToTensor`, which turns the 0–255 pixel values into a $1 \times 28 \times 28$ float tensor in $[0, 1]$, and `Normalize`, which subtracts the mean and divides by the standard deviation. I computed the mean (0.1307) and standard deviation (0.3082) from the 48,000 training images only, so the validation and test images do not influence the preprocessing. No data augmentation is used. Normalized inputs are centered around zero, which keeps the first layer's outputs in a reasonable range at the start of training.

2.2 RANDOM SEED AND IDENTICAL STARTING CONDITIONS

All experiments use the seed 42. `set_seed` (Listing 2.2) seeds Python, NumPy and PyTorch and turns on PyTorch's deterministic algorithms. It is called before the network is created, so every configuration starts from the same initial weights. The training loader shuffles with its own generator, also seeded with 42, so all runs see the batches in the same order. Data loading uses `num_workers=0`, which keeps everything in one process.

```
def set_seed(seed):
    """Seed every random number generator used during training."""
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.use_deterministic_algorithms(True)
```

Listing 2.2. Seeding every random number generator (`train.py`).

3 TASK 1 — CNN IMPLEMENTATION AND MODEL DOCUMENTATION

Task 1 asks for the CNN implementation in PyTorch, the purpose and input/output dimensions of each layer, a table of trainable parameters with an explanation of how they were counted, and the training configuration with a short justification. Section 3.1 covers the layers, Section 3.2 the parameters, Section 3.3 the configuration, and Section 3.4 the training loop and how the metrics are calculated.

3.1 ARCHITECTURE AND LAYER DIMENSIONS

The network follows the LeNet-5 design (LeCun et al., 1998): two convolutional layers, each followed by ReLU and 2×2 max pooling, and then three fully connected layers (Listing 3.1). This meets the minimum requirement of two convolutional layers, one pooling layer and one fully connected layer.

```
class Net(nn.Module):
    """Two convolutional layers, two max-pooling layers and three fully
        connected layers.

    Shapes for one 1 x 28 x 28 MNIST image:
        conv1 (5x5, padding 2) -> 6 x 28 x 28 -> max-pool 2x2 -> 6 x 14 x
            14
        conv2 (5x5)            -> 16 x 10 x 10 -> max-pool 2x2 -> 16 x 5 x
             5
        flatten                -> 400
        fc1 -> 120, fc2 -> 84, fc3 -> 10 class scores (logits)
    """

    def __init__(self):
        super(Net, self).__init__()
        # LAYERS
        self.conv1 = nn.Conv2d(1, 6, 5, padding=2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)
```

```
19
20      def forward(self, x):
21          # Feed-forward
22          x = F.max_pool2d(F.relu(self.conv1(x)), 2)
23          x = F.max_pool2d(F.relu(self.conv2(x)), 2)
24          x = torch.flatten(x, 1)
25          x = F.relu(self.fc1(x))
26          x = F.relu(self.fc2(x))
27          x = self.fc3(x)           # no softmax: CrossEntropyLoss applies
                log-softmax itself
28          return x
```

Listing 3.1. The CNN (`model.py`).

The purpose of each layer, for one input image, is:

- **conv1** ($1 \times 28 \times 28 \to 6 \times 28 \times 28$): six 5×5 kernels learn low-level features such as stroke edges and line ends, the same kind of features the Sobel filter found in Assignment 2. Padding of 2 keeps the size at 28×28 (output size $= 28 + 2 \cdot 2 - 5 + 1 = 28$), so strokes near the border are not lost. The lecture example feeds a $1 \times 32 \times 32$ image to conv1 without padding (Abid, 2026); padding the 28×28 MNIST image by 2 pixels gives the same 32×32 input, so every later layer keeps the lecture's dimensions.
- **pool1** ($6 \times 28 \times 28 \to 6 \times 14 \times 14$): max pooling halves the resolution and keeps the strongest response in every 2×2 block.
- **conv2** ($6 \times 14 \times 14 \to 16 \times 10 \times 10$): sixteen 5×5 kernels, each looking at all six maps from conv1, combine edges into larger parts of digits such as curves, corners and loops. No padding, so $14 - 5 + 1 = 10$.
- **pool2** ($16 \times 10 \times 10 \to 16 \times 5 \times 5$): halves the resolution again.
- **flatten** ($16 \times 5 \times 5 \to 400$): turns the feature maps into one vector for the fully connected layers.
- **fc1** ($400 \to 120$) and **fc2** ($120 \to 84$): combine the features from the whole image to decide which digit it is.
- **fc3** ($84 \to 10$): one score per digit. There is no softmax here because `nn.CrossEntropyLoss` applies log-softmax internally (Equation 1.1).

ReLU is applied after conv1, conv2, fc1 and fc2. ReLU, pooling and flatten have no parameters.

3.2 TRAINABLE PARAMETERS

A convolutional layer with C_{in} input channels, C_{out} kernels of size $k \times k$ has $C_{out}(C_{in}k^2 + 1)$ parameters: each kernel has $C_{in}k^2$ weights plus one bias. A fully connected layer with n_{in} inputs

and n_{out} outputs has $n_{out}(n_{in}+1)$ parameters. Table 3.1 applies these formulas to every layer.

Table 3.1. Output shape and trainable parameters of every layer, for one input image.

Layer	Output shape	Calculation	Parameters
conv1	$6 \times 28 \times 28$	$6 \times (1 \cdot 5 \cdot 5 + 1)$	156
pool1	$6 \times 14 \times 14$	—	0
conv2	$16 \times 10 \times 10$	$16 \times (6 \cdot 5 \cdot 5 + 1)$	2,416
pool2	$16 \times 5 \times 5$	—	0
flatten	400	—	0
fc1	120	$120 \times (400 + 1)$	48,120
fc2	84	$84 \times (120 + 1)$	10,164
fc3	10	$10 \times (84 + 1)$	850
Total			**61,706**

To check the hand calculation, `model.py` also counts the parameters in code: `layer_table` passes a dummy image through the network with forward hooks to record each layer's output shape, and sums `p.numel()` over each layer's tensors. The program prints the same 61,706 total (Listing 3.2). Almost 78% of the parameters are in fc1, while the two convolutional layers together hold only about 4%, because a convolution reuses the same small kernel at every position of the image.

```
=== Layers (input -> output shape, trainable parameters)
conv1   (1, 28, 28)      -> (6, 28, 28)          156
conv2   (6, 14, 14)      -> (16, 10, 10)        2416
fc1     (400,)           -> (120,)             48120
fc2     (120,)           -> (84,)              10164
fc3     (84,)            -> (10,)                850
total trainable parameters: 61706
```

Listing 3.2. Program output: layer shapes and parameter counts.

3.3 TRAINING CONFIGURATION

Justification. Cross-entropy is the standard loss for single-label classification with 10 classes, and it is the loss used in the lecture's full training example. That example trains with `optim.SGD(lr=0.001, momentum=0.9)` and a batch size of 4 (Abid, 2026). I kept SGD with momentum, which averages recent gradients and so smooths the noisy mini-batch updates. I used a batch of 64, which makes each gradient estimate less noisy and an epoch much faster (750 updates instead of 12,000). With 16 times fewer updates per epoch, a learning rate of 0.001 would move slowly, so I raised it to 0.01 for the baseline, the value the lecture uses in its parameter-update example (Abid, 2026). Task 4 (Chapter 6) tests whether this was a good choice. I left out dropout and weight decay so that the baseline shows the plain learning behavior of the network, and any overfitting would be visible in the curves.

Table 3.2. Baseline training configuration.

Setting	Value
Optimizer	SGD with momentum 0.9
Learning rate	0.01 (constant, no scheduler)
Batch size	64 (750 batches per epoch)
Epochs	10
Loss function	Cross-entropy (`nn.CrossEntropyLoss`)
Weight initialization	PyTorch default, drawn right after seeding
Random seed	42 (split, initial weights, shuffling)
Preprocessing	ToTensor, then Normalize(mean 0.1307, std 0.3082)
Regularization	None (no dropout, no weight decay)

3.4 TRAINING LOOP AND METRIC CALCULATION

Listing 3.3 shows the training loop. For every batch it clears the old gradients, runs the forward pass, computes the loss, back-propagates and updates the weights. During the first epoch it also records, for each batch, the loss, the accuracy and the macro-F1 of that batch's predictions, plus a cumulative macro-F1 over every image seen so far in the epoch. These are the values plotted in Task 2 (Chapter 4).

```
    for epoch in range(1, cfg["epochs"] + 1):
        model.train()
        start = time.time()
        seen_true, seen_pred = [], []
        for i, (images, labels) in enumerate(trainloader, start=1):
            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

            if epoch == 1:
                # batch-level record for Task 2: metrics of this batch's own predictions
                preds = outputs.argmax(1)
                seen_true.append(labels)
                seen_pred.append(preds)
                batch_hist["batch"].append(i)
                batch_hist["loss"].append(loss.item())
                batch_hist["acc"].append((preds == labels).float().mean().item())
                batch_hist["f1"].append(macro_f1(labels.numpy(), preds.numpy()))
                # cumulative macro-F1 over every training image seen so far in epoch 1
                batch_hist["f1_cum"].append(macro_f1(torch.cat(seen_true).numpy(),
                                                     torch.cat(seen_pred).
```

```
                                                    numpy()))

        train_time = time.time() - start
        # epoch-level summaries: both sets are re-evaluated in eval mode
            with the end-of-epoch weights
        tr = evaluate(model, trainloader, criterion)
        va = evaluate(model, valloader, criterion)
        for key, value in [("epoch", epoch), ("train_loss", tr["loss"]), (
            "train_acc", tr["acc"]),
                           ("train_f1", tr["f1"]), ("val_loss", va["loss"
                               ]), ("val_acc", va["acc"]),
                           ("val_f1", va["f1"]), ("time", train_time)]:
            epoch_hist[key].append(value)
        log(f"epoch {epoch:2d} | train loss {tr[`loss`]:.4f} acc {tr[`acc`
            ]:.4f} F1 {tr[`f1`]:.4f} "
            f"| val loss {va[`loss`]:.4f} acc {va[`acc`]:.4f} F1 {va[`f1`
                ]:.4f} | {train_time:.1f}s")
```

Listing 3.3. Training loop with first-epoch batch logging (`train.py`).

At the end of every epoch, `evaluate` (Listing 3.4) runs the model in evaluation mode, without gradients, over the whole training split and the whole validation split. I measure the training metrics this way instead of averaging the batch values during the epoch, because the batch values mix early and late weights from the same epoch. Re-evaluating both sets with the same end-of-epoch weights makes the training and validation numbers directly comparable, which matters for the gap analysis in Task 3 (Chapter 5).

Every F1-score in this report is macro-F1 (Equation 1.3), computed by `macro_f1` with scikit-learn's `f1_score(average="macro")`: the F1-score of each digit is computed separately and the values are averaged with equal weight over the digits that appear in the labels or the predictions. On the full training, validation and test sets that is always all 10 digits. Macro-precision and macro-recall for Task 5 are computed the same way with `precision_score` and `recall_score`.

```
def macro_f1(y_true, y_pred):
    # Unweighted mean of the per-class F1-scores over the digits that
        appear in y_true or y_pred.
    # On the full training/validation/test sets this is all 10 digits; a
        64-image batch can miss
    # a digit, and scoring that absent digit as F1 = 0 would wrongly pull
        the batch's score down.
    return f1_score(y_true, y_pred, average="macro", zero_division=0)


def evaluate(model, loader, criterion):
    """Loss, accuracy, macro-F1 and the raw predictions over a whole
        loader (eval mode, no gradients)."""
    model.eval()
    total_loss, y_true, y_pred = 0.0, [], []
    with torch.no_grad():
        for images, labels in loader:
            outputs = model(images)
            total_loss += criterion(outputs, labels).item() * labels.size
                (0)
```

```
            y_true.append(labels)
            y_pred.append(outputs.argmax(1))
    y_true = torch.cat(y_true).numpy()
    y_pred = torch.cat(y_pred).numpy()
    return {
        "loss": total_loss / len(y_true),
        "acc": float((y_true == y_pred).mean()),
        "f1": float(macro_f1(y_true, y_pred)),
        "y_true": y_true,
        "y_pred": y_pred,
    }
```

Listing 3.4. Macro-F1 and the evaluation function (`train.py`).

4 TASK 2 — FIRST-EPOCH BATCH-LEVEL LEARNING ANALYSIS

Task 2 asks for the training loss, training accuracy and F1-score of the baseline plotted by batch during the first epoch (2–4 graphs), an explanation of the learning behavior, and how the F1-score was calculated. The baseline was trained for the full 10 epochs; this chapter looks only at epoch 1, and Chapter 5 covers all 10.

During the first epoch of the baseline (750 batches of 64 images), the training loop recorded the loss, accuracy and macro-F1 of every batch (Section 3.4). To keep the plots readable, the faint line shows only every 5th batch, and the solid line is a 50-batch running average over all batches. The per-batch macro-F1 is computed on the 64 predictions of that batch alone, with scikit-learn's `f1_score(average="macro")`, averaging over the digits present in that batch's labels or predictions. A random batch of 64 occasionally misses a digit entirely (8 of the 750 batches in this epoch), and that digit is left out of the average instead of being counted as an F1 of 0. Since a batch has only about six images of each digit, one mistake changes a class's F1 by a lot, so Figure 4.3 also shows a cumulative macro-F1 computed over every image seen so far in the epoch.

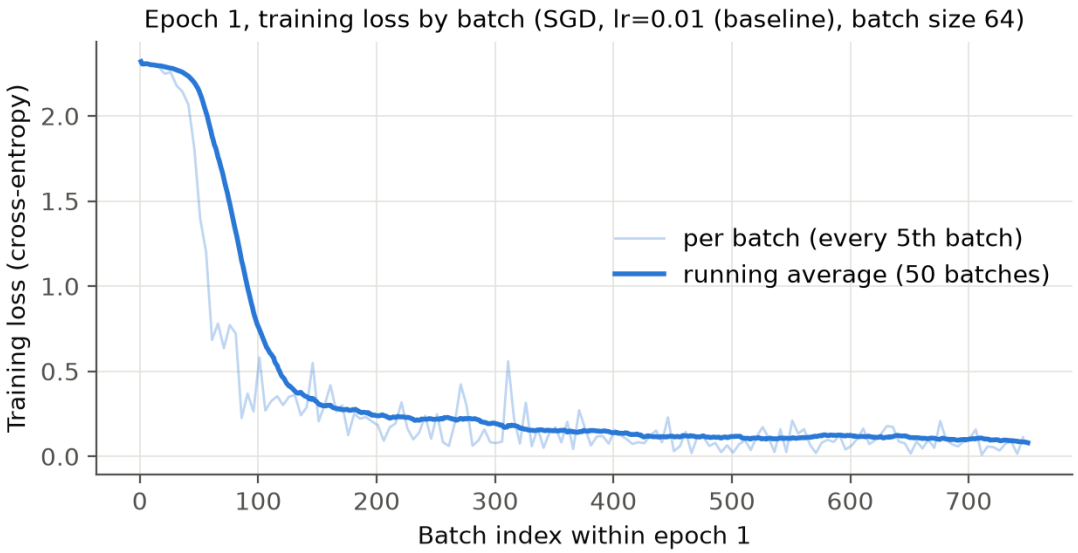


Figure 4.1. Baseline (SGD, $\eta = 0.01$, batch size 64): training cross-entropy loss during the first epoch.

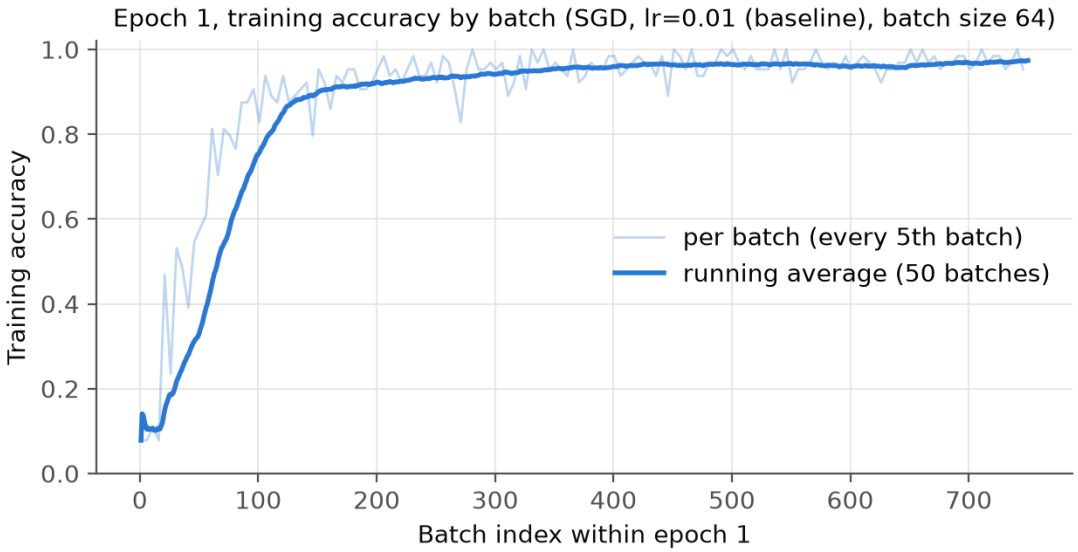


Figure 4.2. Baseline: training accuracy of each batch during the first epoch.

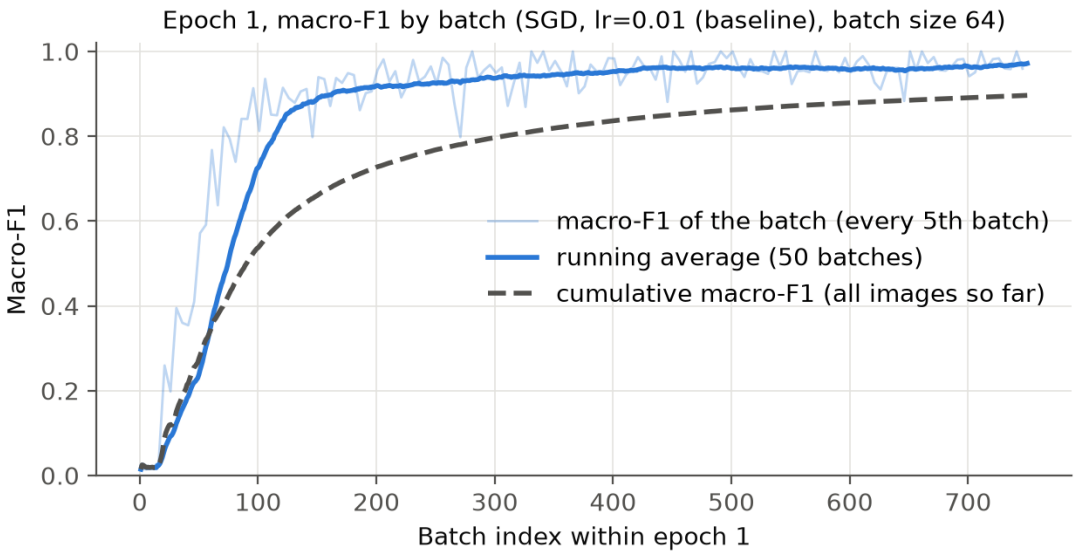


Figure 4.3. Baseline: per-batch macro-F1, its running average, and the cumulative macro-F1 over all images seen so far in the first epoch.

Observation. The first batch has a loss of 2.32 and an accuracy of 7.8%. This is what an untrained network should give: with random weights, all 10 classes get about the same probability, so the loss is close to $\ln 10 = 2.30$ and accuracy is close to the 10% chance level. The epoch then goes through three phases:

1. Slow start (batches 1–40). The loss only goes from 2.32 to 2.04. The gradients are small at first, and momentum needs a few steps to build up speed.

2. Fast learning (batches 40–150). The loss falls quickly: the 50-batch average is 2.15 at batch 50, 0.76 at batch 100 and 0.24 at batch 200. Accuracy and macro-F1 rise together, and the running accuracy passes 90% at batch 151. Most of the epoch's improvement happens in these hundred or so batches.

3. Refinement (batches 150–750). Progress slows down and the curves flatten. By the end of the epoch, the running loss is 0.08 and the running accuracy is 97.4%.

The per-batch curves stay noisy until the end (in the last 100 batches, batch accuracy still varies between 92% and 100%) because each batch is a different random sample of 64 images. Accuracy and macro-F1 follow almost the same path: MNIST classes are nearly balanced, so no digit is being ignored and the two metrics agree. The cumulative macro-F1 ends at 0.896, well below the late-batch values of about 0.97, because it still includes the predictions from the start of the epoch when the network was guessing.

5 TASK 3 — TEN-EPOCH TRAINING AND VALIDATION ANALYSIS

Task 3 asks for the training and validation loss and accuracy of the baseline over the 10 epochs, the epoch-level macro-F1, and a discussion of convergence, the training–validation gap and any over- or underfitting. Only epoch-level summaries are used here.

At the end of every epoch, the training split (48,000 images) and the validation split (12,000 images) were evaluated with the same weights in evaluation mode (Section 3.4). Table 5.1 lists the values, and Figures 5.1–5.3 plot them.

Table 5.1. Baseline (SGD, $\eta = 0.01$, batch size 64): epoch-level loss, accuracy and macro-F1 on the training and validation splits.

Epoch	Train loss	Train acc.	Train F1	Val. loss	Val. acc.	Val. F1
1	0.0903	97.23%	0.9722	0.1084	96.75%	0.9673
2	0.0575	98.14%	0.9812	0.0724	97.78%	0.9776
3	0.0431	98.61%	0.9859	0.0693	97.83%	0.9781
4	0.0413	98.67%	0.9866	0.0643	98.27%	0.9826
5	0.0261	99.16%	0.9915	0.0557	98.31%	0.9829
6	0.0227	99.28%	0.9927	0.0574	98.29%	0.9827
7	0.0136	99.58%	0.9958	**0.0425**	**98.77%**	**0.9876**
8	0.0131	99.59%	0.9959	0.0473	98.63%	0.9862
9	0.0248	99.11%	0.9909	0.0558	98.54%	0.9853
10	0.0129	99.56%	0.9956	0.0473	98.68%	0.9867

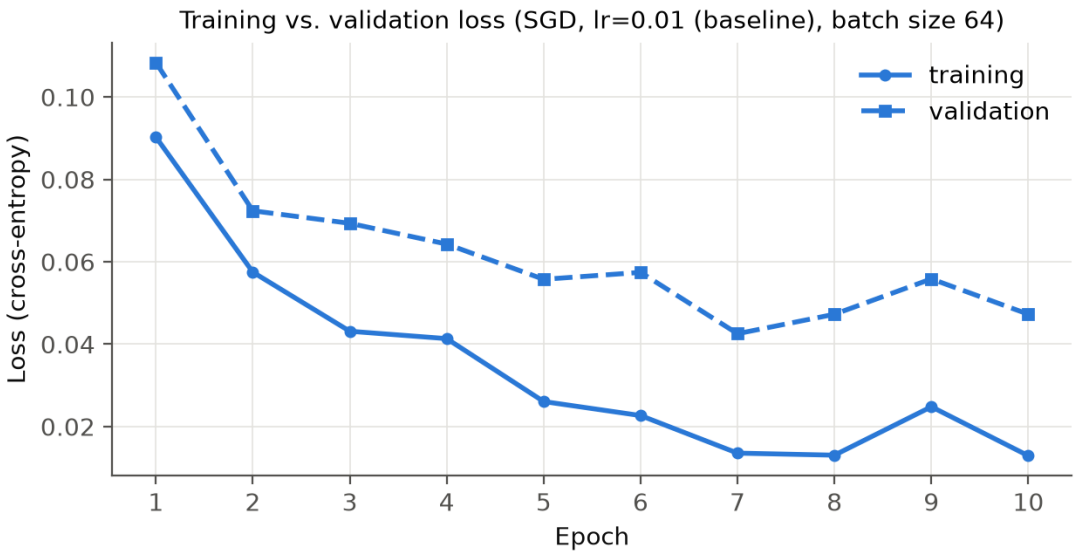


Figure 5.1. Baseline: training and validation cross-entropy loss per epoch.

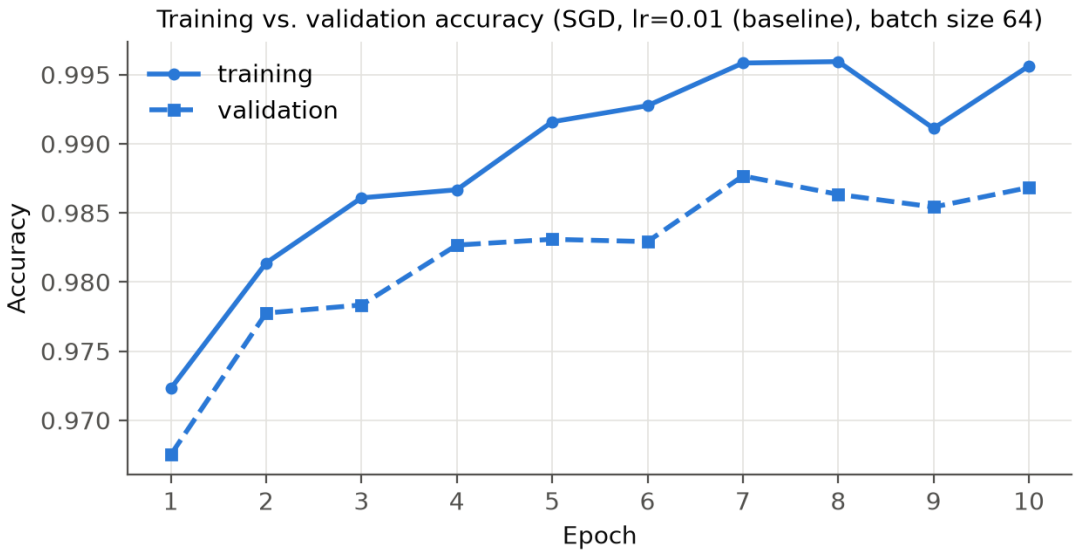


Figure 5.2. Baseline: training and validation accuracy per epoch.

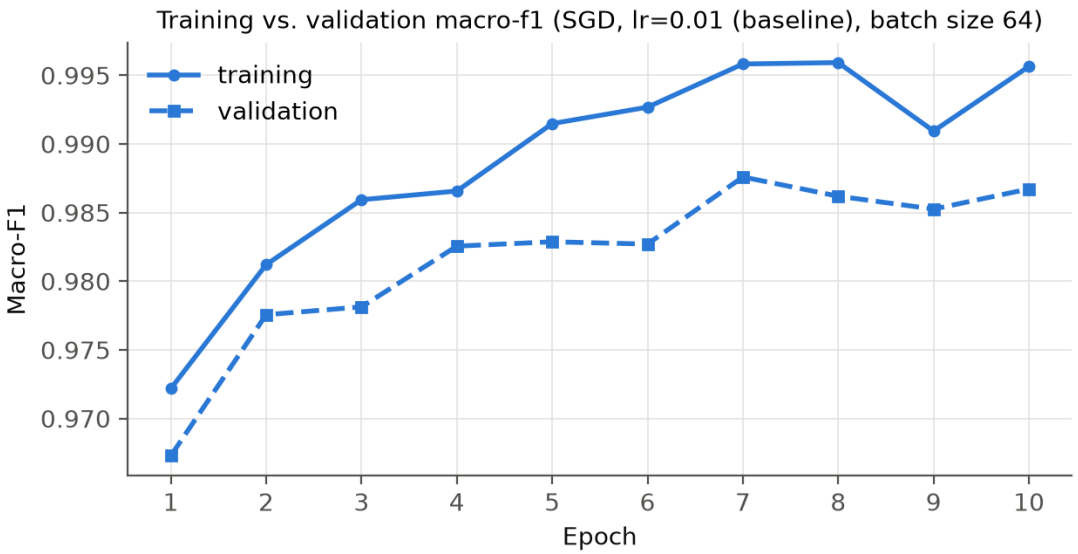


Figure 5.3. Baseline: training and validation macro-F1 per epoch.

Convergence. Most of the learning happens in the first epoch: after it, validation accuracy is already 96.75%. The next epochs add smaller gains, and validation accuracy stays between 98.27% and 98.77% from epoch 4 on. The best validation epoch is epoch 7 (loss 0.0425, accuracy 98.77%). After that the validation curves stop improving and move up and down within about 0.25 percentage points, so the model has converged by about epoch 7. Epoch 9 shows a small bump: training loss rises from 0.0131 to 0.0248 and validation loss from 0.0473 to 0.0558, and both recover in epoch 10. With a constant learning rate and momentum, a few unlucky batches can push the weights slightly away from a good point. The training and validation curves move together here, so I read it as optimization noise. Overfitting would show up as validation loss rising while training loss falls.

Training–validation gap. Training is better than validation at every epoch, and the gap grows over time. In accuracy it goes from 0.48 percentage points at epoch 1 to 0.88 points at epoch 10; in loss, the training loss at epoch 10 (0.0129) is less than a third of the validation loss (0.0473). Macro-F1 follows accuracy almost exactly on both sets (differences of at most 0.0002), so the errors are spread across the digits.

Overfitting and underfitting. There is no underfitting: the network reaches 99.6% training accuracy, so it has enough capacity for this task. There is *mild* overfitting after epoch 7: the training loss keeps going down while the validation loss stops improving (0.0425 at epoch 7, then 0.047–0.056), so the extra epochs mostly fit the training images more tightly. The effect is small, though. Validation accuracy does not drop in any meaningful way (98.68% at epoch 10 against 98.77% at its best), and the final model still generalizes well. Dropout or weight decay could reduce the gap further, and early stopping around epoch 7 would be another option.

6 TASK 4 — CONTROLLED LEARNING-RATE EXPERIMENT

Task 4 asks me to choose one factor, compare the baseline with at least one alternative under otherwise fixed settings, train each for 10 epochs, compare their curves, and explain which configuration I would select and why. Section 6.1 describes the setup and Section 6.2 the results and the selection.

6.1 EXPERIMENT DESIGN

The factor I chose to study is the learning rate. Besides the baseline ($\eta = 0.01$), I trained two alternatives, one ten times smaller ($\eta = 0.001$) and one ten times larger ($\eta = 0.1$). Everything else is identical to Table 3.2: same optimizer and momentum, same batch size, same split, same seed, same initial weights and same batch order. The three configurations are listed at the top of `main.py` (Listing 6.1).

```
SEED = 42
BASE = {"optimizer": "SGD", "momentum": 0.9, "batch_size": 64, "epochs":
    10, "seed": SEED}

# Task 4 factor: learning rate. Everything else stays the same as the
    baseline.
CONFIGS = [
    {**BASE, "name": "lr_0.01", "lr": 0.01, "baseline": True},
    {**BASE, "name": "lr_0.001", "lr": 0.001},
    {**BASE, "name": "lr_0.1", "lr": 0.1},
]
BASELINE = "lr_0.01"
```

Listing 6.1. The three configurations (`main.py`).

Selection rule. The rule was fixed before looking at any results: the configuration with the highest validation accuracy after epoch 10 is selected, with lower validation loss as the tie-breaker. The test set is only loaded after this choice is made, and only once.

6.2 RESULTS AND SELECTION

Figure 6.1 shows the training and validation curves side by side, Figure 6.2 compares the first epoch, Figure 6.3 zooms in on validation accuracy, and Table 6.1 summarizes the results.

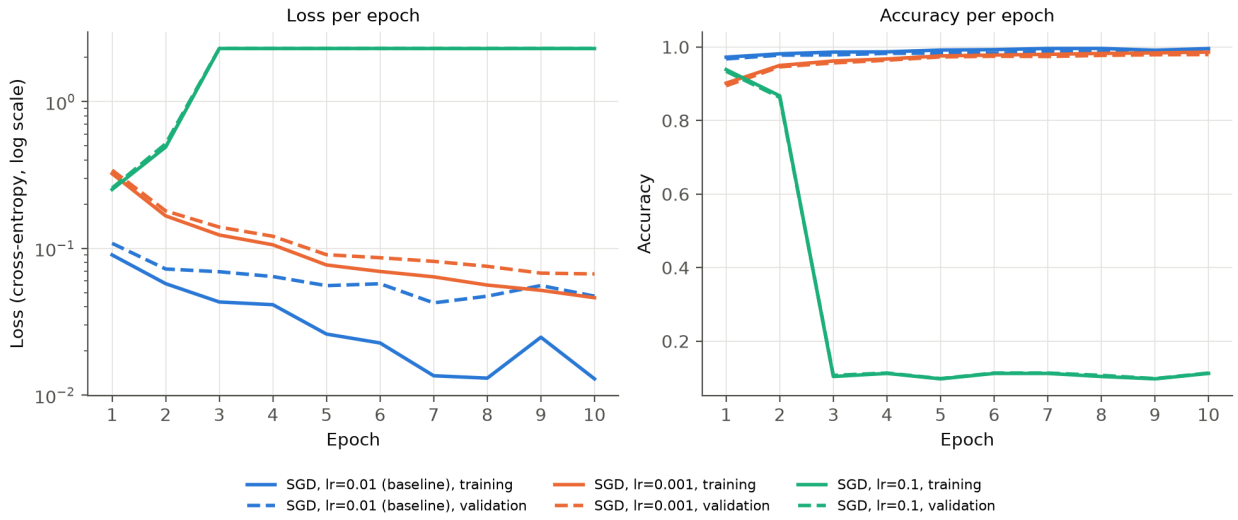


Figure 6.1. Training (solid) and validation (dashed) loss and accuracy per epoch for the three learning rates (SGD, momentum 0.9, batch size 64). The loss axis is logarithmic.

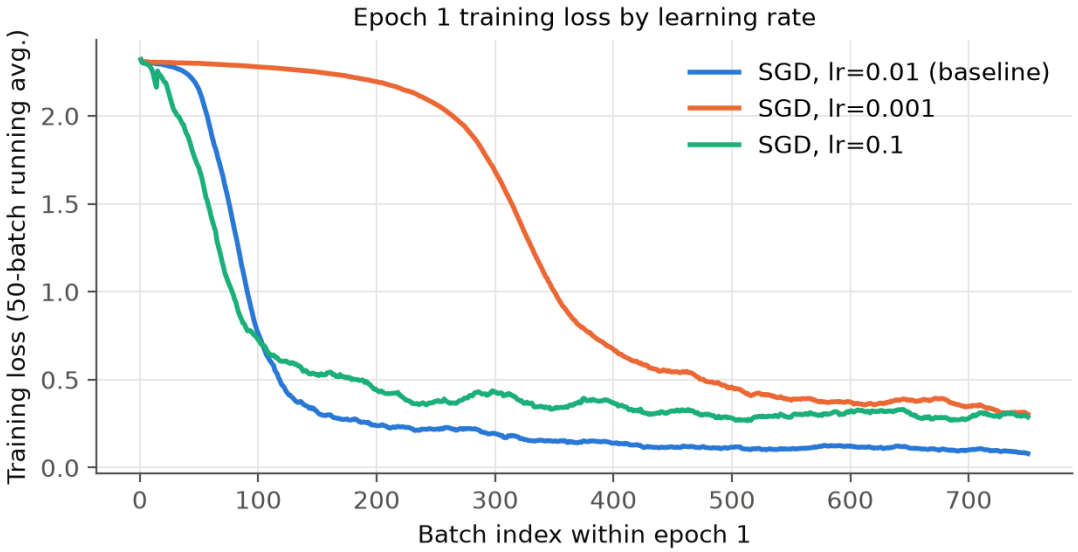


Figure 6.2. Training loss (50-batch running average) during the first epoch for the three learning rates.

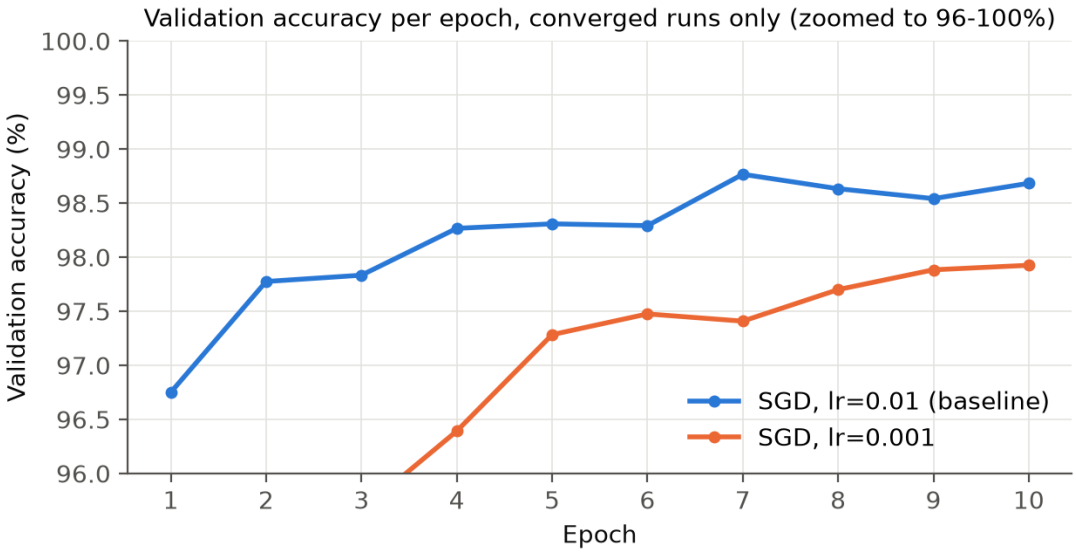


Figure 6.3. Validation accuracy per epoch for $\eta = 0.01$ and $\eta = 0.001$, zoomed to 96–100%. The $\eta = 0.1$ run is left out because it stays near 11% after epoch 3.

Table 6.1. Learning-rate comparison. Final values are after epoch 10; the gap is training accuracy minus validation accuracy at epoch 10.

	$\eta = 0.001$	$\eta = 0.01$ (baseline)	$\eta = 0.1$
Val. accuracy after epoch 1	89.44%	96.75%	93.38%
Batch where running acc. passes 90%	714	151	228
Final training accuracy	98.60%	99.56%	11.22%
Final validation accuracy	97.92%	**98.68%**	11.33%
Final validation macro-F1	0.9790	**0.9867**	0.0203
Final validation loss	0.0670	**0.0473**	2.3030
Best validation accuracy (epoch)	97.92% (10)	98.77% (7)	93.38% (1)
Train–val. accuracy gap	0.67 pts	0.88 pts	—

Small learning rate ($\eta = 0.001$): too slow. With steps ten times smaller, the network stays on the initial plateau much longer. In the first epoch the loss is still 2.19 after 200 batches, and the running accuracy passes 90% only at batch 714, almost five times later than the baseline (Figure 6.2). After one epoch it has 89.4% validation accuracy, against 96.8% for the baseline. It keeps improving steadily and never shows instability, but after 10 epochs it reaches only 97.92%, about where the baseline was after epoch 3. Its validation loss is still going down at epoch 10, so this run has not converged yet: it is slightly *underfitting* within the 10-epoch budget. Its train–validation gap is the smallest (0.67 points), but only because it has not fit the training data as tightly yet.

Large learning rate ($\eta = 0.1$): fast, then unstable. At first the large steps help: in Figure 6.2 its loss falls earlier than the baseline's. But the curve is noisier, with a loss spike to 3.55 at batch

15, and in the second half of the epoch its loss stays higher (0.30 on average, against 0.11 for the baseline), because the steps are too big to settle into a good minimum. After epoch 1 its validation accuracy is 93.4%. In epoch 2 it gets worse (86.3%), and in epoch 3 training collapses. From then on the loss stays at 2.303, which is exactly $\ln 10$, and accuracy stays at about 11%: the network has collapsed to predicting the same digit for every image, which is why its validation macro-F1 is only 0.0203. Training and validation accuracy are equally bad, which points to a failure of the optimization itself.

Selection. Following the rule set in Section 6.1, the selected configuration is the **baseline,** $\eta = 0.01$. It has the highest final validation accuracy (98.68%), the highest validation macro-F1 (0.9867) and the lowest validation loss (0.0473). It also converges much faster than $\eta = 0.001$ and is stable, while $\eta = 0.1$ diverges. Its generalization gap (0.88 points) is a little larger than that of $\eta = 0.001$, but its validation accuracy is still 0.76 points higher. With more epochs $\eta = 0.001$ might catch up. Within the 10 epochs required here, 0.01 is the better choice.

7 TASK 5 — INDEPENDENT TESTING AND REPRODUCIBILIT

Task 5 asks for a single evaluation of the selected configuration on the reserved MNIST test set, reporting accuracy, macro-precision, macro-recall and macro-F1, together with the random seed, the library versions, the execution environment and instructions to reproduce the experiments. These are covered in Sections 7.1, 7.2 and 7.3.

7.1 TEST-SET EVALUATION

Only after the configuration was selected did `main.py` load the 10,000 test images, evaluating the saved epoch-10 weights of the baseline once. Listing 7.1 shows the program output and Table 7.1 the metrics.

```
=== Validation summary after epoch 10
lr_0.01   val acc 0.9868  val F1 0.9867  val loss 0.0473  best val acc
   0.9877 (epoch 7)
lr_0.001  val acc 0.9792  val F1 0.9790  val loss 0.0670  best val acc
   0.9792 (epoch 10)
lr_0.1    val acc 0.1133  val F1 0.0203  val loss 2.3030  best val acc
   0.9338 (epoch 1)
selected configuration: lr_0.01

=== TEST (lr_0.01, evaluated once)
test_loss        0.0402
test_acc         0.9884
macro_precision  0.9882
macro_recall     0.9885
macro_f1         0.9883
```

Listing 7.1. Program output: model selection and the single test evaluation.

Observation. The model classifies 9,884 of the 10,000 test images correctly, so it makes 116 errors. The test accuracy (98.84%) is close to the validation accuracy (98.68%), and slightly higher. The validation set was therefore a good estimate of performance on unseen data, and selecting on it did not overfit to the validation images. Macro-precision, macro-recall and macro-F1 are all within 0.0003 of the accuracy, so no digit is much worse than the others.

Table 7.1. Test-set results of the selected model (SGD, $\eta = 0.01$, batch size 64, weights after epoch 10, seed 42).

Metric	Test set (10,000 images)
Accuracy	98.84%
Macro-precision	0.9882
Macro-recall	0.9885
Macro-F1	0.9883
Cross-entropy loss	0.0402

7.2 EXECUTION ENVIRONMENT AND LIBRARY VERSIONS

Table 7.2. Software and hardware environment used for every experiment.

Component	Version	Purpose
Python	3.13.5	Programming language
PyTorch	2.14.1 (CPU build)	Tensors, layers, autograd, optimizer (Paszke et al., 2019)
torchvision	0.29.1	MNIST download and transforms
scikit-learn	1.9.1	Precision, recall and F1 (Pedregosa et al., 2011)
NumPy / Matplotlib	2.5.4 / 3.11.2	Arrays and figures (Hunter, 2007)
Operating system	Windows 11 Home	64-bit
Hardware	Intel Core i7-13700F, 16 GB RAM	All training on the CPU (16 threads); no GPU used

The computer has an NVIDIA RTX 4060 Ti, but I installed the CPU build of PyTorch on purpose. The network is small enough to train on the CPU in a few minutes, and on the CPU `torch.use_deterministic_algorithms(True)` makes every run give the same numbers, which is harder to guarantee with GPU kernels.

7.3 INSTRUCTIONS TO REPRODUCE THE EXPERIMENTS

All results in this report were produced by the code in Appendix A on the machine in Table 7.2. To reproduce them:

1. Install Python 3.13 and create a virtual environment in the project folder:

```
python -m venv env
env\Scripts\activate          (Windows)
source env/bin/activate       (Linux / macOS)
```

2. Install the pinned dependencies (CPU build of PyTorch):

```
pip install -r requirements.txt
```

3. Run all experiments. MNIST is downloaded to `./data` on the first run (internet needed once):

```
python main.py
```

   This trains the three configurations (about 10 minutes in total on the CPU above), selects one using the validation set, evaluates it once on the test set, and writes:

   - `results/output_log.txt`: the full program output;
   - `results/<config>/history.json` and `model.pt`: metrics and final weights per configuration;
   - `results/test_metrics.json`: the test metrics;
   - `report/figures/*.png`: every figure in this report.

4. To redo only the selection, the test evaluation and the figures from saved results, run `python main.py -skip-train`.

Random seed. Seed 42 is used for the train/validation split, the initial weights and the shuffling order (Section 2.2). Training runs on the CPU with `torch.use_deterministic_algorithms(True)` and `num_workers=0`, so running `main.py` again on the same software versions gives the same numbers. I checked this by training all three configurations a second time: every epoch-level loss, accuracy and F1-score, and the final weights, matched the first run exactly. Different PyTorch versions or a GPU may change the last digits because of floating-point differences.

8 CONCLUSION

In this assignment I implemented a LeNet-style CNN in PyTorch with two convolutional layers, two pooling layers and three fully connected layers (61,706 parameters), and trained it on a fixed 48,000/12,000 split of MNIST. During the first epoch, the model went from chance level to over 97% accuracy. Over 10 epochs it converged by about epoch 7, with mild overfitting after that. In the learning-rate experiment, 0.001 was too slow to converge in 10 epochs and 0.1 collapsed in epoch 3, so the baseline of 0.01 was selected on the validation set. Evaluated once on the 10,000 test images, it reached 98.84% accuracy, 0.9882 macro-precision, 0.9885 macro-recall and 0.9883 macro-F1.

Limitations. Each configuration was trained with a single seed, so differences of a few tenths of a percent between runs should be read with care. Only three learning rates, a factor of 10 apart, were tested, and the reported model uses the weights from the end of epoch 10, even though epoch 7 had a slightly better validation score.

REFERENCES

Abid, M. 2026. "Lecture 08: Deep learning libraries and PyTorch." *CAP 5410 Advanced Computer Vision*, *lecture notes*, *Florida Polytechnic University*.

Goodfellow, I., Bengio, Y., and Courville, A. 2016. *Deep Learning*. MIT Press.

Hunter, J. D. 2007. "Matplotlib: A 2D graphics environment." *Computing in Science & Engineering*, 9 (3): 90–95.

K., H. 2017. "MNIST dataset." *Kaggle*. Accessed through `torchvision.datasets.MNIST`, October 2026.

LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P. 1998. "Gradient-based learning applied to document recognition." *Proceedings of the IEEE*, 86 (11): 2278–2324.

Paszke, A. et al. 2019. "PyTorch: An imperative style, high-performance deep learning library." *Advances in Neural Information Processing Systems 32*, 8024–8035.

Pedregosa, F. et al. 2011. "Scikit-learn: Machine learning in Python." *Journal of Machine Learning Research*, 12: 2825–2830.

PyTorch Team 2026. "Training a classifier (PyTorch tutorials)." *PyTorch documentation*. Accessed October 2026.

A SOURCE CODE

The complete source files are listed below; they are the same files submitted alongside this report.

A.1 DATALOADER.PY

```
"""CAP 5410 - Assignment 3: MNIST data loading.

Downloads MNIST through torchvision, splits the 60,000 training images
into 48,000 for training and 12,000 for validation with a fixed seed,
and keeps the 10,000 test images apart for the final evaluation.
"""
import torch
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader, Subset

DATA_DIR = "./data"
SPLIT_SEED = 42          # same split for every experiment
N_TRAIN = 48_000
N_VAL = 12_000


def split_indices(seed=SPLIT_SEED):
    """Return (train_idx, val_idx): a fixed random 80/20 split of the
       60,000 images."""
    g = torch.Generator().manual_seed(seed)
    perm = torch.randperm(N_TRAIN + N_VAL, generator=g)
    return perm[:N_TRAIN].tolist(), perm[N_TRAIN:].tolist()


def train_mean_std(train_idx):
    """Mean and standard deviation of the pixels in the training split
       only.

    The validation and test images are not used here, so no information
    about them leaks into the preprocessing.
    """
```

```
    raw = torchvision.datasets.MNIST(root=DATA_DIR, train=True, download=True)
    pixels = raw.data[train_idx].float() / 255.0
    return pixels.mean().item(), pixels.std().item()


def get_loaders(batch_size, seed=SPLIT_SEED):
    """Build the train, validation and test DataLoaders.

    Preprocessing: ToTensor (0-255 -> 0-1) followed by normalization with
    the training-split mean and std. Only the training loader is shuffled,
    and its shuffle order comes from a seeded generator so that every run
    sees the batches in the same order.
    """
    train_idx, val_idx = split_indices(seed)
    mean, std = train_mean_std(train_idx)

    transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((mean,), (std,)),
    ])

    full_train = torchvision.datasets.MNIST(root=DATA_DIR, train=True, download=True, transform=transform)
    testset = torchvision.datasets.MNIST(root=DATA_DIR, train=False, download=True, transform=transform)

    trainset = Subset(full_train, train_idx)
    valset = Subset(full_train, val_idx)

    g = torch.Generator().manual_seed(seed)
    trainloader = DataLoader(trainset, batch_size=batch_size, shuffle=True, generator=g, num_workers=0)
    valloader = DataLoader(valset, batch_size=1000, shuffle=False, num_workers=0)
    testloader = DataLoader(testset, batch_size=1000, shuffle=False, num_workers=0)

    return trainloader, valloader, testloader, (mean, std)
```

Listing A.1. MNIST loading and data split (`dataloader.py`).

A.2 MODEL.PY

```
"""CAP 5410 - Assignment 3: CNN model (LeNet-5 style) for MNIST."""
import torch
import torch.nn as nn
import torch.nn.functional as F


class Net(nn.Module):
```

```
    """Two convolutional layers, two max-pooling layers and three fully
       connected layers.

    Shapes for one 1 x 28 x 28 MNIST image:
        conv1 (5x5, padding 2) -> 6 x 28 x 28 -> max-pool 2x2 -> 6 x 14 x
           14
        conv2 (5x5)            -> 16 x 10 x 10 -> max-pool 2x2 -> 16 x 5 x
            5
        flatten                -> 400
        fc1 -> 120, fc2 -> 84, fc3 -> 10 class scores (logits)
    """

    def __init__(self):
        super(Net, self).__init__()
        # LAYERS
        self.conv1 = nn.Conv2d(1, 6, 5, padding=2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        # Feed-forward
        x = F.max_pool2d(F.relu(self.conv1(x)), 2)
        x = F.max_pool2d(F.relu(self.conv2(x)), 2)
        x = torch.flatten(x, 1)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)          # no softmax: CrossEntropyLoss applies
           log-softmax itself
        return x


def layer_table(model, input_shape=(1, 1, 28, 28)):
    """Return one row per layer: name, output shape and trainable
       parameters.

    Output shapes are found by passing a dummy image through the network
    with forward hooks; parameter counts come from the layer's tensors.
    """
    rows = []
    hooks = []

    def hook(name):
        def fn(module, inp, out):
            n_params = sum(p.numel() for p in module.parameters() if p.
               requires_grad)
            rows.append((name, tuple(inp[0].shape[1:]), tuple(out.shape
               [1:]), n_params))
        return fn

    for name, module in model.named_children():
        hooks.append(module.register_forward_hook(hook(name)))
    with torch.no_grad():
```

```
        model(torch.zeros(input_shape))
    for h in hooks:
        h.remove()
    return rows


def count_parameters(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)
```

Listing A.2. CNN and parameter table (`model.py`).

A.3 TRAIN.PY

```
"""CAP 5410 - Assignment 3: training and evaluation loops.

`run_experiment` trains one configuration for a fixed number of epochs and
returns everything the report needs: batch-level metrics for the first
epoch and epoch-level metrics for the training and validation sets.
"""
import json
import os
import random
import time

import numpy as np
import torch
import torch.nn as nn
from sklearn.metrics import f1_score, precision_score, recall_score
from torch.utils.data import DataLoader

from dataloader import get_loaders
from model import Net

RESULTS_DIR = "./results"


def set_seed(seed):
    """Seed every random number generator used during training."""
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.use_deterministic_algorithms(True)


def macro_f1(y_true, y_pred):
    # Unweighted mean of the per-class F1-scores over the digits that
        appear in y_true or y_pred.
    # On the full training/validation/test sets this is all 10 digits; a
        64-image batch can miss
    # a digit, and scoring that absent digit as F1 = 0 would wrongly pull
        the batch's score down.
```

```
    return f1_score(y_true, y_pred, average="macro", zero_division=0)


def evaluate(model, loader, criterion):
    """Loss, accuracy, macro-F1 and the raw predictions over a whole loader (eval mode, no gradients)."""
    model.eval()
    total_loss, y_true, y_pred = 0.0, [], []
    with torch.no_grad():
        for images, labels in loader:
            outputs = model(images)
            total_loss += criterion(outputs, labels).item() * labels.size(0)
            y_true.append(labels)
            y_pred.append(outputs.argmax(1))
    y_true = torch.cat(y_true).numpy()
    y_pred = torch.cat(y_pred).numpy()
    return {
        "loss": total_loss / len(y_true),
        "acc": float((y_true == y_pred).mean()),
        "f1": float(macro_f1(y_true, y_pred)),
        "y_true": y_true,
        "y_pred": y_pred,
    }


def make_optimizer(name, params, lr, momentum):
    if name == "SGD":
        return torch.optim.SGD(params, lr=lr, momentum=momentum)
    if name == "Adam":
        return torch.optim.Adam(params, lr=lr)
    raise ValueError(f"unknown optimizer {name}")


def run_experiment(cfg, log=print):
    """Train `cfg` and save its history and final weights under results/<name>."""
    set_seed(cfg["seed"])
    trainloader, valloader, _, (mean, std) = get_loaders(cfg["batch_size"], cfg["seed"])
    # unshuffled copy of the training split, used only to measure epoch-level training metrics
    train_eval_loader = DataLoader(trainloader.dataset, batch_size=1000, shuffle=False)

    model = Net()            # created right after seeding: same initial weights for every config
    criterion = nn.CrossEntropyLoss()
    optimizer = make_optimizer(cfg["optimizer"], model.parameters(), cfg["lr"], cfg["momentum"])

    log(f"\n=== {cfg['name']}: {cfg['optimizer']} lr={cfg['lr']} momentum={cfg['momentum']} "
```

```
        f"batch={cfg['batch_size']} epochs={cfg['epochs']} seed={cfg['seed']}")
    log(f"normalization (training split): mean={mean:.4f} std={std:.4f}")

    batch_hist = {"batch": [], "loss": [], "acc": [], "f1": [], "f1_cum": []}
    epoch_hist = {k: [] for k in ["epoch", "train_loss", "train_acc", "train_f1",
                                  "val_loss", "val_acc", "val_f1", "time"]}

    for epoch in range(1, cfg["epochs"] + 1):
        model.train()
        start = time.time()
        seen_true, seen_pred = [], []
        for i, (images, labels) in enumerate(trainloader, start=1):
            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

            if epoch == 1:
                # batch-level record for Task 2: metrics of this batch's own predictions
                preds = outputs.argmax(1)
                seen_true.append(labels)
                seen_pred.append(preds)
                batch_hist["batch"].append(i)
                batch_hist["loss"].append(loss.item())
                batch_hist["acc"].append((preds == labels).float().mean().item())
                batch_hist["f1"].append(macro_f1(labels.numpy(), preds.numpy()))
                # cumulative macro-F1 over every training image seen so far in epoch 1
                batch_hist["f1_cum"].append(macro_f1(torch.cat(seen_true).numpy(),
                                                     torch.cat(seen_pred).numpy()))

        train_time = time.time() - start
        # epoch-level summaries: both sets are re-evaluated in eval mode with the end-of-epoch weights
        tr = evaluate(model, trainloader, criterion)
        va = evaluate(model, valloader, criterion)
        for key, value in [("epoch", epoch), ("train_loss", tr["loss"]), ("train_acc", tr["acc"]),
                           ("train_f1", tr["f1"]), ("val_loss", va["loss"]), ("val_acc", va["acc"]),
                           ("val_f1", va["f1"]), ("time", train_time)]:
            epoch_hist[key].append(value)
        log(f"epoch {epoch:2d} | train loss {tr['loss']:.4f} acc {tr['acc']:.4f} F1 {tr['f1']:.4f} "
```

```
            f"| val loss {va['loss']:.4f} acc {va['acc']:.4f} F1 {va['f1']:.4f} | {train_time:.1f}s")

    out_dir = os.path.join(RESULTS_DIR, cfg["name"])
    os.makedirs(out_dir, exist_ok=True)
    torch.save(model.state_dict(), os.path.join(out_dir, "model.pt"))
    with open(os.path.join(out_dir, "history.json"), "w") as f:
        json.dump({"config": cfg, "norm": [mean, std], "batch": batch_hist, "epoch": epoch_hist}, f, indent=1)
    return model


def test_model(cfg, log=print):
    """Evaluate the saved weights of `cfg` once on the 10,000 MNIST test images."""
    _, _, testloader, _ = get_loaders(cfg["batch_size"], cfg["seed"])
    model = Net()
    model.load_state_dict(torch.load(os.path.join(RESULTS_DIR, cfg["name"], "model.pt"), weights_only=True))
    res = evaluate(model, testloader, nn.CrossEntropyLoss())
    y_true, y_pred = res["y_true"], res["y_pred"]
    metrics = {
        "config": cfg["name"],
        "test_loss": res["loss"],
        "test_acc": res["acc"],
        "macro_precision": float(precision_score(y_true, y_pred, average="macro", zero_division=0)),
        "macro_recall": float(recall_score(y_true, y_pred, average="macro", zero_division=0)),
        "macro_f1": res["f1"],
        "per_class_f1": f1_score(y_true, y_pred, average=None).tolist(),
    }
    log(f"\n=== TEST ({cfg['name']}, evaluated once)")
    for k in ["test_loss", "test_acc", "macro_precision", "macro_recall", "macro_f1"]:
        log(f"{k:16s} {metrics[k]:.4f}")
    with open(os.path.join(RESULTS_DIR, "test_metrics.json"), "w") as f:
        json.dump(metrics, f, indent=1)
    np.save(os.path.join(RESULTS_DIR, "test_predictions.npy"), np.stack([y_true, y_pred]))
    return metrics
```

Listing A.3. Training, evaluation and testing (`train.py`).

A.4 MAIN.PY

```
"""CAP 5410 - Assignment 3: run every experiment end to end.

    python main.py              train all configurations, pick one on validation, test it once, make figures
```

```
    python main.py --skip-train reuse the saved results/ and only redo
        selection, test and figures

Steps:
  1. Print the environment and the per-layer parameter table.
  2. Train the baseline and the learning-rate alternatives for 10 epochs
      each.
  3. Select the configuration with the best final validation accuracy
      (the test set is not touched until this choice is made).
  4. Evaluate the selected model once on the 10,000 test images.
  5. Save every figure into report/figures/.
"""
import argparse
import json
import os
import platform
import sys

import matplotlib
import numpy as np
import sklearn
import torch
import torchvision

import plots
from model import Net, count_parameters, layer_table
from train import RESULTS_DIR, run_experiment, test_model

SEED = 42
BASE = {"optimizer": "SGD", "momentum": 0.9, "batch_size": 64, "epochs":
    10, "seed": SEED}

# Task 4 factor: learning rate. Everything else stays the same as the
    baseline.
CONFIGS = [
    {**BASE, "name": "lr_0.01", "lr": 0.01, "baseline": True},
    {**BASE, "name": "lr_0.001", "lr": 0.001},
    {**BASE, "name": "lr_0.1", "lr": 0.1},
]
BASELINE = "lr_0.01"


class Logger:
    """Print to the console and keep a copy in results/output_log.txt."""

    def __init__(self, path, mode):
        os.makedirs(os.path.dirname(path), exist_ok=True)
        self.f = open(path, mode, encoding="utf-8")

    def __call__(self, msg=""):
        print(msg)
        self.f.write(msg + "\n")
        self.f.flush()

```

```
54
55 def environment(log):
56     log("=== Environment")
57     log(f"Python       {sys.version.split()[0]}")
58     log(f"PyTorch      {torch.__version__}")
59     log(f"torchvision  {torchvision.__version__}")
60     log(f"NumPy        {np.__version__}")
61     log(f"scikit-learn {sklearn.__version__}")
62     log(f"Matplotlib   {matplotlib.__version__}")
63     log(f"OS           {platform.platform()}")
64     log(f"Processor    {platform.processor()}")
65     log(f"Device       CPU (CUDA available: {torch.cuda.is_available()}),
           {torch.get_num_threads()} threads")
66     log(f"Seed         {SEED}")
67
68
69 def parameter_table(log):
70     log("\n=== Layers (input -> output shape, trainable parameters)")
71     model = Net()
72     for name, inp, out, n in layer_table(model):
73         log(f"{name:6s} {str(inp):15s} -> {str(out):15s} {n:7d}")
74     log(f"total trainable parameters: {count_parameters(model)}")
75
76
77 def select(log):
78     """Pick the configuration with the highest validation accuracy after
           epoch 10 (validation data only)."""
79     log("\n=== Validation summary after epoch 10")
80     best, best_key = None, None
81     for cfg in CONFIGS:
82         with open(os.path.join(RESULTS_DIR, cfg["name"], "history.json"))
               as f:
83             e = json.load(f)["epoch"]
84         key = (e["val_acc"][-1], -e["val_loss"][-1])     # tie-break on
               lower validation loss
85         log(f"{cfg[`name`]:9s} val acc {e[`val_acc`][-1]:.4f}  val F1 {e[`
               val_f1`][-1]:.4f}  "
86             f"val loss {e[`val_loss`][-1]:.4f}  best val acc {max(e[`
                   val_acc`]):.4f} "
87             f"(epoch {int(np.argmax(e[`val_acc`])) + 1})")
88         if best is None or key > best_key:
89             best, best_key = cfg, key
90     log(f"selected configuration: {best[`name`]}")
91     return best
92
93
94 def main():
95     parser = argparse.ArgumentParser()
96     parser.add_argument("--skip-train", action="store_true", help="reuse
           saved histories and weights")
97     args = parser.parse_args()
98
99     log = Logger(os.path.join(RESULTS_DIR, "output_log.txt"), "a" if args.
           skip_train else "w")
```

```
    environment(log)
    parameter_table(log)

    if not args.skip_train:
        for cfg in CONFIGS:
            run_experiment(cfg, log)

    chosen = select(log)
    test_model(chosen, log)
    plots.make_all([c["name"] for c in CONFIGS], BASELINE)
    log(f"\nfigures saved to {plots.FIG_DIR}")


if __name__ == "__main__":
    main()
```

Listing A.4. Experiment driver (`main.py`).

A.5 PLOTS.PY

```
"""CAP 5410 - Assignment 3: figures for the report.

Reads results/<config>/history.json and
writes every figure into report/figures/.
"""
import json
import os

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

RESULTS_DIR = "./results"
FIG_DIR = "./report/figures"

# fixed colors per configuration (color follows the config, never its rank)
COLORS = {"lr_0.01": "#2a78d6", "lr_0.001": "#eb6834", "lr_0.1": "#1baf7a"}
INK = "#52514e"

plt.rcParams.update({
    "figure.dpi": 150, "savefig.dpi": 200, "font.size": 10,
    "axes.edgecolor": INK, "axes.labelcolor": "#0b0b0b",
    "axes.spines.top": False, "axes.spines.right": False,
    "axes.grid": True, "grid.color": "#e4e3df", "grid.linewidth": 0.6,
    "xtick.color": INK, "ytick.color": INK, "legend.frameon": False,
    "lines.linewidth": 2,
})

```

```
30
31 def load(name):
32     with open(os.path.join(RESULTS_DIR, name, "history.json")) as f:
33         return json.load(f)
34
35
36 def label(h):
37     c = h["config"]
38     return f"{c['optimizer']}, lr={c['lr']}" + (" (baseline)" if c.get("
           baseline") else "")
39
40
41 def moving_average(x, w=50):
42     """Trailing moving average; the first w-1 points average whatever is
           available."""
43     x = np.asarray(x, dtype=float)
44     c = np.cumsum(np.insert(x, 0, 0.0))
45     out = np.empty_like(x)
46     for i in range(len(x)):
47         lo = max(0, i + 1 - w)
48         out[i] = (c[i + 1] - c[lo]) / (i + 1 - lo)
49     return out
50
51
52 def save(fig, name):
53     os.makedirs(FIG_DIR, exist_ok=True)
54     fig.tight_layout()
55     fig.savefig(os.path.join(FIG_DIR, name), bbox_inches="tight")
56     plt.close(fig)
57
58
59 # ---------------------------------------------------------------------
60 # Task 2: first epoch, batch level (baseline)
61 # ---------------------------------------------------------------------
62
63 def plot_first_epoch(h, step=5, window=50):
64     b = h["batch"]
65     color = COLORS[h["config"]["name"]]
66     batches = np.array(b["batch"])
67     bs = h["config"]["batch_size"]
68     cfg_txt = f"{label(h)}, batch size {bs}"
69     specs = [
70         ("loss", "Training loss (cross-entropy)", "epoch1_loss.png"),
71         ("acc", "Training accuracy", "epoch1_accuracy.png"),
72     ]
73     for key, ylabel, fname in specs:
74         fig, ax = plt.subplots(figsize=(6.4, 3.4))
75         ax.plot(batches[::step], np.array(b[key])[::step], color=color,
               alpha=0.3, linewidth=1,
76                 label=f"per batch (every {step}th batch)")
77         ax.plot(batches, moving_average(b[key], window), color=color,
78                 label=f"running average ({window} batches)")
79         ax.set_xlabel("Batch index within epoch 1")
80         ax.set_ylabel(ylabel)
```

```
        if key == "acc":
            ax.set_ylim(0, 1.02)
        ax.set_title(f"Epoch 1, {ylabel.split(' (')[0].lower()} by batch ({cfg_txt})", fontsize=10)
        ax.legend(loc="center right")
        save(fig, fname)

    fig, ax = plt.subplots(figsize=(6.4, 3.4))
    ax.plot(batches[::step], np.array(b["f1"])[::step], color=color, alpha=0.3, linewidth=1,
            label=f"macro-F1 of the batch (every {step}th batch)")
    ax.plot(batches, moving_average(b["f1"], window), color=color,
            label=f"running average ({window} batches)")
    ax.plot(batches, b["f1_cum"], color=INK, linestyle="--",
            label="cumulative macro-F1 (all images so far)")
    ax.set_xlabel("Batch index within epoch 1")
    ax.set_ylabel("Macro-F1")
    ax.set_ylim(0, 1.02)
    ax.set_title(f"Epoch 1, macro-F1 by batch ({cfg_txt})", fontsize=10)
    ax.legend(loc="center right")
    save(fig, "epoch1_f1.png")


# ---------------------------------------------------------------------
# Task 3: ten epochs, epoch level (baseline)
# ---------------------------------------------------------------------

def plot_epochs(h):
    e = h["epoch"]
    color = COLORS[h["config"]["name"]]
    for key, ylabel, fname in [("loss", "Loss (cross-entropy)", "epochs_loss.png"),
                               ("acc", "Accuracy", "epochs_accuracy.png"),
                               ("f1", "Macro-F1", "epochs_f1.png")]:
        fig, ax = plt.subplots(figsize=(6.4, 3.4))
        ax.plot(e["epoch"], e[f"train_{key}"], color=color, marker="o", markersize=4, label="training")
        ax.plot(e["epoch"], e[f"val_{key}"], color=color, marker="s", markersize=4,
                linestyle="--", label="validation")
        ax.set_xlabel("Epoch")
        ax.set_ylabel(ylabel)
        ax.set_xticks(e["epoch"])
        ax.set_title(f"Training vs. validation {ylabel.split(' (')[0].lower()} ({label(h)}, "
                     f"batch size {h[`config`][`batch_size`]})", fontsize=10)
        ax.legend()
        save(fig, fname)


# ---------------------------------------------------------------------
# Task 4: learning-rate comparison
# ---------------------------------------------------------------------
```

```
128
129 def plot_comparison(histories):
130     fig, axes = plt.subplots(1, 2, figsize=(10, 3.8))
131     for key, ax, ylabel in [("loss", axes[0], "Loss (cross-entropy)"), ("
        acc", axes[1], "Accuracy")]:
132         for h in histories:
133             e = h["epoch"]
134             color = COLORS[h["config"]["name"]]
135             ax.plot(e["epoch"], e[f"train_{key}"], color=color, label=f"{
                label(h)}, training")
136             ax.plot(e["epoch"], e[f"val_{key}"], color=color, linestyle="
                --",
137                     label=f"{label(h)}, validation")
138         ax.set_xlabel("Epoch")
139         ax.set_ylabel(ylabel)
140         ax.set_xticks(range(1, 11))
141         ax.set_title(f"{ylabel.split(' (')[0]} per epoch", fontsize=10)
142     axes[0].set_yscale("log")
143     axes[0].set_ylabel("Loss (cross-entropy, log scale)")
144     handles, labels = axes[0].get_legend_handles_labels()
145     fig.legend(handles, labels, loc="lower center", ncol=3, fontsize=8,
        bbox_to_anchor=(0.5, -0.12))
146     save(fig, "compare_curves.png")
147
148     # validation accuracy zoomed in, only for the runs that converged (a
        diverged run sits near 10%)
149     fig, ax = plt.subplots(figsize=(6.4, 3.4))
150     for h in histories:
151         e = h["epoch"]
152         if e["val_acc"][-1] < 0.5:
153             continue
154         ax.plot(e["epoch"], np.array(e["val_acc"]) * 100, color=COLORS[h["
            config"]["name"]],
155                 marker="o", markersize=4, label=label(h))
156     ax.set_xlabel("Epoch")
157     ax.set_ylabel("Validation accuracy (%)")
158     ax.set_xticks(range(1, 11))
159     ax.set_ylim(96, 100)
160     ax.set_title("Validation accuracy per epoch, converged runs only (
        zoomed to 96-100%)", fontsize=10)
161     ax.legend(loc="lower right")
162     save(fig, "compare_val_acc.png")
163
164     # first-epoch training loss for every configuration
165     fig, ax = plt.subplots(figsize=(6.4, 3.4))
166     for h in histories:
167         b = h["batch"]
168         ax.plot(b["batch"], moving_average(b["loss"], 50), color=COLORS[h[
            "config"]["name"]], label=label(h))
169     ax.set_xlabel("Batch index within epoch 1")
170     ax.set_ylabel("Training loss (50-batch running avg.)")
171     ax.set_title("Epoch 1 training loss by learning rate", fontsize=10)
172     ax.legend()
173     save(fig, "compare_epoch1_loss.png")
```

```
174
175
176 def make_all(names, baseline):
177     histories = [load(n) for n in names]
178     base = load(baseline)
179     plot_first_epoch(base)
180     plot_epochs(base)
181     plot_comparison(histories)
182
183
184 if __name__ == "__main__":
185     from main import CONFIGS, BASELINE
186     make_all([c["name"] for c in CONFIGS], BASELINE)
```

Listing A.5. Figures (`plots.py`).